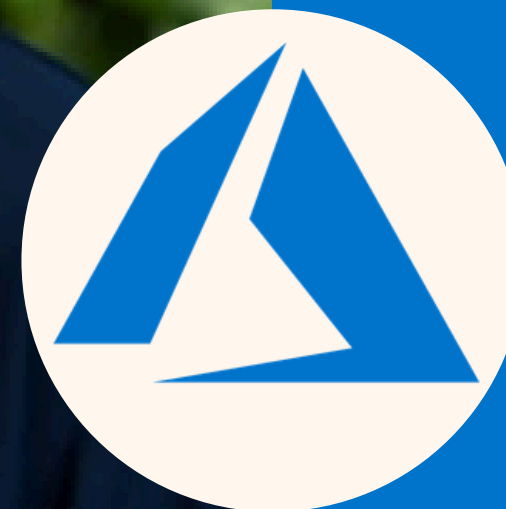


 **GET STARTED TODAY**

LUKE

GINN



**AZURE
CLOUD
MASTERY**



Unit 17 covered multimodal inputs. Unit 18 covers input transformation – using NLP services to clean and structure user prompts before they reach the agent core.

- **Review of Unit 17:** You learned to process images, audio, and documents. Now you need to clean messy text inputs – typos, slang, ambiguous phrases – before the agent reasons about them.
- **What This Unit Covers:** You will learn Azure AI Language services for entity extraction (find people, dates, places), key phrase extraction, sentiment analysis, and PII detection.
- **Why Input Transformation Matters:** Raw user input is noisy. NLP preprocessing reduces agent confusion, improves intent resolution, and protects sensitive data.

Exam Takeaway: Input Transformation is part of Agentic Enhancement domain (10% weighting). The exam tests calling NLP services before routing to agent core.



What Is Input Transformation?

Input transformation is the process of applying NLP (Natural Language Processing) services to user input before sending it to the agent's LLM or orchestration logic.

- **Input Transformation Definition:** Raw user text → NLP service → cleaned, structured, enriched text → agent core. The agent sees improved input.
- **Why Transform Input:** Users make typos ("wether" instead of "weather"), use vague language ("that thing"), or accidentally include sensitive data (credit card numbers).
- **Transformation Happens BEFORE Agent:** The agent core never sees raw input. It only sees the transformed version. This improves reliability and security.

Exam Takeaway: Input transformation is a preprocessing layer before the agent. The exam tests placing transformation before agent core, not after.



Azure AI Language is a suite of NLP services that extract entities, detect sentiment, identify key phrases, and redact personally identifiable information (PII).

- **Service Components:** Language includes Entity Recognition (find people, dates, locations), Sentiment Analysis (positive/negative/neutral), Key Phrase Extraction, and PII Detection.
- **Why Use Language:** Instead of building your own spell checker or entity extractor, Language provides pre-trained models ready to use via API.
- **Integration Pattern:** Agent receives user message → calls Language API → receives enriched JSON → passes transformed text to LLM or agent orchestrator.

Exam Takeaway: Azure AI Language is the primary NLP preprocessing service. The exam tests which Language feature to use for different transformation needs.



Named Entity Recognition (NER) scans text and identifies entities like person names, dates, locations, organizations, phone numbers, and email addresses.

- **NER Definition:** NER tags each entity with a category (Person, Date, Location, Organization, PhoneNumber, Email, URL, etc.) and its position in the text.
- **Agent Use Case:** User says "Book a flight to Seattle on May 15th." NER extracts city = "Seattle", date = "May 15th". Agent uses extracted values directly.
- **Reducing Ambiguity:** Instead of agent parsing "Seattle" as a city or a person, NER confirms it is a Location entity. Improves tool call accuracy.

Exam Takeaway: NER extracts structured entities from unstructured text. The exam tests using NER to populate tool parameters (dates, locations, names).



Key phrase extraction identifies the most important concepts and topics in a user's message, ignoring filler words and irrelevant details.

- **Key Phrase Definition:** The service returns a list of phrases that summarize the main topics. Example: "My laptop screen cracked after I dropped it" → key phrases: ["laptop screen", "cracked", "dropped it"].
- **Agent Use Case:** User writes a long, rambling complaint. Key phrase extraction extracts core issues. Agent focuses on those instead of reading every word.
- **Reducing Noise:** Key phrases filter out conversational filler ("um", "well", "you know") leaving only meaningful content for the agent.

Exam Takeaway: Key phrase extraction summarizes long text. The exam tests when to use key phrases (long, noisy inputs) vs. full text (short, clear inputs).



Sentiment Analysis – Detecting User Emotion

Sentiment analysis determines whether a user's message is positive, negative, neutral, or mixed, with confidence scores for each sentiment.

- **Sentiment Definition:** Service returns `sentiment` (positive, negative, neutral, mixed) and confidence scores (0-1) for positive, negative, and neutral.
- **Agent Use Case:** Negative sentiment (0.9 negative, 0.1 positive) → escalate to human agent immediately. Positive sentiment → route to automated responses.
- **Mixed Sentiment:** Message contains both positive ("fast shipping") and negative ("broken product"). Sentiment returns "mixed". Agent reads full text instead of relying on sentiment.

Exam Takeaway: Sentiment analysis routes users based on emotion. The exam tests using sentiment scores to decide escalation vs. automation.



PII detection scans text for personally identifiable information – credit card numbers, social security numbers, bank accounts – and optionally redacts them.

- **PII Categories:** Detects credit card numbers, Social Security numbers, bank account numbers, driver's license numbers, passport numbers, email addresses, phone numbers, and addresses.
- **Redaction Option:** Replace detected PII with `[REDACTED\_CREDIT\_CARD]` or `[REDACTED\_EMAIL]`. The agent never sees the actual sensitive data.
- **Use Case:** User types credit card number in chat. PII detection redacts it. Agent processes the request without storing or logging the sensitive number.

Exam Takeaway: PII detection protects sensitive data before it reaches the agent. The exam tests redacting PII before logging or LLM processing.



Language Detection – Identifying User's Language

Language detection identifies which language the user is writing in (English, Spanish, French, etc.), allowing the agent to respond in the same language.

- **Language Detection Definition:** Service analyzes text and returns language code (e.g., `en`, `es`, `fr`) and confidence score (0-1).
- **Agent Use Case:** Spanish-speaking user writes in Spanish. Agent detects `es`, translates system instructions to Spanish, calls Spanish-speaking tools, responds in Spanish.
- **Confidence Threshold:** If confidence < 0.8, agent may ask "What language are you speaking?" or default to English. Avoids incorrect language detection.

Exam Takeaway: Language detection enables multilingual agents. The exam tests using detected language to select language-specific system instructions.



Your agent code calls Azure AI Language using the SDK or REST, sending text and requesting specific capabilities (entities, key phrases, sentiment, PII, language).

- **SDK Pattern:** ``from azure.ai.textanalytics import TextAnalyticsClient``.
``client.recognize_entities([user_message])[0]`` returns list of entities.
- **Multiple Capabilities:** Call once with ``client.analyze_sentiment()`` for sentiment, ``client.extract_key_phrases()`` for key phrases, or batch calls for efficiency.
- **Response Handling:** Extract ``entities`` array (each has ``text``, ``category``, ``confidence_score``). Extract ``sentiment`` and ``confidence_scores``. Extract ``key_phrases`` list.

Exam Takeaway: Call Language API with user message. Parse JSON response for entities, sentiment, key phrases, and PII. Handle each based on use case.



Transforming Input with Extracted Entities

After calling NER, your code replaces raw text references with structured entity tags, making it easier for the agent to understand parameters.

- **Transformation Pattern:** Raw: "Book flight to Seattle on May 15th." Transformed: "Book flight to [LOCATION: Seattle] on [DATE: May 15th]."
- **Entity Replacement:** For each detected entity, replace the original text span with `[CATEGORY: value]`. Agent sees structured format.
- **Benefits:** Agent does not need to parse "Seattle" as a city – the NER tag tells it directly. Reduces tool call errors.

Exam Takeaway: Replace raw entities with structured tags. The exam tests transforming user input into agent-friendly structured format.



Routing Based on Sentiment

Use sentiment scores to route user messages to different agent workflows – immediate escalation for negative sentiment, automation for positive/neutral sentiment.

- **Negative Sentiment Routing:** If ``sentiment == "negative"`` and ``confidence_scores.negative > 0.8``, route directly to human support agent. Skip automated processing.
- **Positive Sentiment Routing:** If ``sentiment == "positive"``, route to automation but include sentiment in prompt: "User is happy. Maintain positive tone."
- **Mixed Sentiment:** If ``sentiment == "mixed"``, agent reads full text and does not rely on sentiment for routing. Mixed requires human judgment.

Exam Takeaway: Sentiment routing decides automation vs. escalation. The exam tests thresholds for routing based on confidence scores.



Redacting PII Before Logging and LLM

Your agent code must redact PII from user messages before sending to LLM, logging to Foundry Trace, or storing in long-term memory.

- **Redaction API:** ``client.recognize_pii_entities([user_message])[0]``. Extract ``redacted_text`` where sensitive data is replaced with ``[REDACTED]``.
- **LLM Safety:** Send ``redacted_text`` to LLM, not raw user input. LLM never sees credit card numbers. Reduces compliance risk.
- **Memory Storage:** Store redacted text in Cosmos DB or Redis. Never store raw PII. Comply with data protection regulations (GDPR, CCPA).

Exam Takeaway: PII redaction must happen before LLM calls, logging, and storage. The exam tests redacting PII at the input transformation stage.



For multilingual agents, detect user language first, then select language-appropriate system instructions and tool responses.

- **Detection Flow:** Agent receives user message. Calls language detection:
``detected_language = client.detect_language([user_message])`
`[0].primary_language.iso6391_name``.
- **System Instruction Selection:** Load system instruction from language-specific file:
``system_prompt = load_prompt(detected_language)``. Example: ``system_es.txt`` for Spanish.
- **Response Translation:** Agent generates response in detected language. If tools return English-only data, agent translates before sending to user.

Exam Takeaway: Language detection selects language-appropriate system instructions. The exam tests loading language-specific prompts based on detection.



A complete input transformation pipeline processes user input through multiple steps: language detection, PII redaction, entity extraction, and key phrase extraction.

- **Step 1 – Language Detection:** Detect user language. If not supported (confidence < 0.7), default to English and notify user.
- **Step 2 – PII Redaction:** Redact credit cards, SSNs, bank accounts. Store original only if compliance allows; otherwise discard.
- **Step 3 – Entity Extraction:** Extract dates, locations, people, organizations. Replace with structured tags `[DATE: value]`.
- **Step 4 – Key Phrase Extraction:** Extract main topics. Append to prompt as "User mentioned these key topics: [list]".

Exam Takeaway: The pipeline order matters: language first (to select models), then PII (to protect data), then entities, then key phrases.



Foundry Trace records each input transformation step as a separate span, showing what changes were made to user input before the agent core.

- **Language Detection Span:** Span name `"lang_detection"`. Attributes: ``detected_language``, ``confidence_score``, ``duration_ms``. Shows which language handler was selected.
- **PII Redaction Span:** Span name `"pii_redaction"`. Attributes: ``pii_categories_found`` (list), ``redacted_text_length``, ``duration_ms``. Shows what was redacted.
- **Entity Extraction Span:** Span name `"entity_extraction"`. Attributes: ``entity_count``, ``entity_categories`` (e.g., `["Person", "Location", "DateTime"]`), ``duration_ms``.
- **Comparison:** Compare raw input and transformed input spans to verify redaction and tagging worked correctly.

Exam Takeaway: Trace each transformation step with spans. The exam tests using trace attributes to debug transformation failures.



Unit 18 Summary – Input Transformation Mastery

You have learned to preprocess user input with Azure AI Language services – entity extraction, sentiment analysis, PII redaction, language detection, and key phrase extraction.

- **Key Transformations:** NER (extract entities), Sentiment (detect emotion), PII (redact sensitive data), Language Detection (multilingual routing), Key Phrases (summarize).
- **Pipeline Order:** Language detection → PII redaction → Entity extraction → Key phrase extraction → Agent core. The exam tests this order.
- **Coding Skills:** Call Language API with SDK or REST. Parse JSON for entities, sentiment, key phrases, redacted text. Replace entities with structured tags.
- **Observability:** Trace each transformation step with spans recording language, PII categories, entity counts, and durations.
- **Next Unit Preview:** Unit 19 introduces System Instructions – programmatic prompt construction for agent personas, boundaries, and grounding rules.

Exam Takeaway: Input Transformation is 10% of the exam. Master NER for entity extraction, PII redaction for security, sentiment for routing, and pipeline ordering before agent core.

**THANKS
FOR
WATCHING**

